GROUP ASSIGNMENT

TECHNOLOGY PARK MALAYSIA

CT074-3-2

CONCURRENT PROGRAMMING

APD2F2311CS(DA)

HAND OUT DATE : 25 JUNE 2024

HAND IN DATE : 9 JULY 2024

WEIGHTAGE : 25%

INSTRUCTIONS TO CANDIDATES:

1. **Submit your assignment at the administrative counter.**
2. **Students are advised to underpin their answers with the use of references (cited using the Harvard Name System of Referencing).**
3. **Late submission will be awarded zero (0) unless Extenuating Circumstances (EC) are upheld.**
4. **Cases of plagiarism will be penalized.**
5. **The assignment should be bound in an appropriate style (comb bound or stapled).**
6. **Where the assignment submitted in both hardcopy and softcopy, the softcopy of the written assignment and source code (where appropriate) should be on envelope / CD cover and attached to the hardcopy.**
7. **You must obtain 50% overall to pass this module.**
8. **This report is Part 2 of 2 for the assignment.**

Table of Contents

1.0 Introduction 1

2.0 Assumptions 2

3.0 Implementation 2

3.1 Classes 2

3.2 Initialization 4

3.3 Landing 6

3.4 Gate Coasting and Docking 7

3.5 Disembarking 8

3.6 Refuel 9

3.7 Cleaning and Refilling Services 10

3.8 Embark 11

3.9 Undock 11

3.10 Take Off 12

3.11 Statistics 13

4.0 Output 14

4.1 Simulation Process 14

4.2 Emergency Case 15

4.3 Statistics 15

5.0 Conclusion 16

6.0 References 17

Table of Figures

Figure 1- Initialization 4
Figure 2 - ATC Constructor 4
Figure 3 - ATC run 4
Figure 4 - Plane constructor 5
Figure 5 - Plane run 5
Figure 6- Plane requestLand() 6
Figure 7 – ATC confirmLanding 6
Figure 8 - ATC landPlane 7
Figure 9 - Plane requestCoastandDock 7
Figure 10 – ATC ConfirmCoastAndDock 7
Figure 11 – ATC assignGate 8
Figure 12 - Plane requestDisembark 8
Figure 13 - ATC DisembarkPassenger 8
Figure 14 - Passenger Constructor 8
Figure 15 - Passenger disembarkPlane 8
Figure 16 - Action enum class 9
Figure 17 – Passenger's run 9
Figure 18 - Plane RequestRefuel 9
Figure 19 - ATC refuelPlane 9
Figure 20 - RefuellingTruck class 9
Figure 21 - Plane RequestCleanAndRefill 10
Figure 22 - ATC CleaningAndRefilling 10
Figure 23 - Worker Class Cleaning and Refill 10
Figure 24 – Worker's run 11
Figure 25 - ATC EmbarkPassenger 11
Figure 26 - Passenger boardPlane 11
Figure 27 - Plane requestUndock 11
Figure 28 - ATC ConfirmUndock 12
Figure 29 - ATC releaseGate 12
Figure 30 - Plane requestTakeOff 12
Figure 31 - ATC Takeoff 12
Figure 32 - Statistic Variables 13

Figure 33 - Statistics methods 13
Figure 34 - Statistics allGatesEmpty 13
Figure 35 - Statistics printStatistics 13
Figure 36 - Plane Arrive and Request Landing 14
Figure 37 - Coast and Dock 14
Figure 38 - Disembark Passenger 14
Figure 39 - Successful Disembark and Refuel Request 14
Figure 40 - Cleaning and Supplies Refilling 14
Figure 41 - Embark Passengers 14
Figure 42 - Undock 14
Figure 43 - Take Off 15
Figure 44 - Request Landing 15
Figure 45 - Request Accepted and Cut Queue 15
Figure 46 - Land first 15
Figure 47 - Statistics Result 15

1.0 Introduction

The Asia Pacific Airport, a small but busy runway, is the lifeline for regional and international traffic in the area. Given the airport's limited resources and infrastructure, effective operations management is vital. The key difficulty is to coordinate the landing, servicing, and departure of aeroplanes on a single runway while maintaining passenger safety and operating efficiency.

This case study tries to imitate operations at Asia Pacific Airport, with an emphasis on the constraints imposed by the airport's restricted capacity. With only one runway and a maximum of three aircraft permitted on the airport premises at any given time, careful planning and coordination are essential to avoid collisions and assure smooth operations. The simulation will imitate real-world events such as passenger boarding and disembarking, aircraft refuelling, and emergency landings to assess the airport's resilience and efficiency in congested conditions.

The goal of this simulation is twofold: to represent the concurrent and exclusive events that occur during aircraft operations, and to assess the airport's performance by collecting and assessing key metrics. By modelling a congested scenario with six planes attempting to land, including an emergency landing due to a fuel shortage, the study will gain information into the airport's ability to handle high-pressure conditions.

I hope that this simulation will help us identify potential bottlenecks and inefficiencies, as well as provide recommendations for enhancing airport operations. The findings will be critical for airport management, assisting in the creation of measures to increase throughput and provide a consistent experience for both passengers and airlines.

2.0 Assumptions

- After the plane successfully landed, the runway will be available for other plane in queue to land.
- Thread executions, like boarding, disembarking, cleaning, refilling supplies and refuelling occur at a consistent speed and without variability due to external factors.
- Plane marked with an emergency status are prioritized to land first by being placed at the front of the landing queue.
- Waiting times and other metrics are accurately recorded and calculated without accounting for potential data anomalies or outliers.

3.0 Implementation

3.1 Classes

In my system to simulate the operations of Asia Pacific Airport, there are a total of 9 classes, which 8 of them are normal java classes, and one of them is an enum class. The classes works as below:

Classes	Description
AirportSimulator	Initialises and starts the simulation of an airport. It generates instances of ATC, runways, gates, and refuelling semaphores. It then starts the ATC thread and generates numerous Plane threads to mimic plane arrival and management.
ATC	Manages the landing, docking, refuelling, cleaning, and departure of aircraft. It works with several semaphores to manage access to the runway, gates, and refilling trucks. It monitors a landing queue, guarantees that planes land and take off safely, handles passenger disembarkation and embarkation, and provides other ground services.
Plane	Represents a plane in the simulation. Each plane requests a landing, docks at a gate, disembarks and loads passengers, asks refuelling and cleaning, and finally takes off. For these procedures, the plane communicates with ATC and keeps track of its current state and gate information.

Gate	Represents a gate at the airport. It has an ID and keeps track of its available state. Gates are allotted to planes for docking after landing and released when they undock for take-off.
Passenger	Simulates passenger boarding and disembarking from a plane. It simulates the process of passengers getting on and off a plane by doing activities that take a set amount of time each.
RefuellingTruck	Simulates the refuelling of aeroplanes. It locks the refuelling process to ensure that only one plane is refuelled at a time and replicates the time required to refuel.
Worker	Represents a ground service worker who cleans, and refills supplies on a plane. Each worker mimics the time it takes to do a certain action (such as cleaning or replenishing).
Statistics	Tracks and reports simulation statistics, including waiting times, planes served, and passengers boarded. The class provides methods to start and end the simulation, record waiting times, increment counts of planes served and passengers boarded, and print out the collected statistics at the end of the simulation.
Action (enum)	Defines the different actions that can be performed on a plane. It is used to distinguish between the various operations that might be required during the handling of a plane at the airport.

3.2 Initialization

```java
public class AirportSimulator {
    public static void main(String[] args) {
        // Initialize and start the simulation
        ATC atc = new ATC();

        // Creating semaphores for runway, gate and refuel truck to control access
        Semaphore runwaySemaphore = new Semaphore( permits: 1); // Only one plane can take off or land at a time
        Semaphore gateSemaphore = new Semaphore( permits: 3); // Three planes can be docked at gates simultaneously
        Semaphore refuelingSemaphore = new Semaphore( permits: 1); // Only one plane can refuel at a time as there are only one refuel truck

        // Creating a thread for ATC to start the simulation
        Thread atcThread = new Thread( task: atc);
        atcThread.start();

        // For loop to create 6 plane threads and initiate their concurrent processes
        for (int i = 0; i < 6; i++) {
            Plane plane = new Plane( id: "Plane-" + (i+1), atc, runwaySemaphore, gateSemaphore, refuelingSemaphore);
            new Thread( task: plane).start();
            try {
                Thread.sleep( millis: new Random().nextInt( origin: 100, bound: 2000));
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }

        System.out.println("\n===================================== End of Simulation =====================================");
    }
}
```

Figure 1- Initialization

The initialization of the airport simulation starts in the main function of the AirportSimulator class. An ATC runnable object is first created, and a landing queue linked list is generated using the constructor of ATC class to store the planes queuing order.

```java
// Constructor for ATC class
public ATC(){  19 usages

    // Create a landingQueue linked list to store the plane that is waiting in queue for landing
    landingQueue = new LinkedList<Plane>();
}
```

Figure 2 - ATC Constructor

Three semaphores, for runway, gate and refuelling truck are also created to fulfil the basic requirement by controlling access of thread, where only one permit for runway, three permits for gate and one permit for refuelling truck.

```java
// Start the simulation when the atc thread is created and started
public void run() {

    // Start recording the simulation time
    statistic.startSimulation();

    // Main loop to manage plane landings
    while (true) {
        synchronized (landingQueue) {

            // Wait while the landing queue is empty until a plane request for landing
            while (landingQueue.isEmpty()) {
                try {
                    landingQueue.wait();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        }
    }
}
```

Figure 3 - ATC run

After that, the thread of the ATC is created and started to initialize the simulation and record the starting time. Meanwhile when there are no plane arriving, the wait() function is used on the landingQueue to wait for arriving planes notify. In this part of the code, synchronized is used on the landingQueue to ensure that only one thread can access the queue at a time,

preventing multiple access of the code by the multiple plane threads. Then, a for loop is used to create a total of 6 threads for each plane, where the three semaphore and the atc object are brought into the constructor.

```java
// Constructor for Plane class
public Plane(String id, ATC atc, Semaphore runwaySemaphore, Semaphore gateSemaphore, Semaphore refuelingSemaphore) {
    this.id = id;

    // Set Plane-6 to emergency case (Fuel Shortage)
    if (id.equals("Plane-6") ){
        this.emergencyLanding = true;
    }else{
        this.emergencyLanding = false;
    }

    this.landStatus = false;
    this.atc = atc;
    this.runwaySemaphore = runwaySemaphore;
    this.gateSemaphore = gateSemaphore;
    this.refuelingSemaphore = refuelingSemaphore;

    // Randomize number of passengers to disembark from and embark onto each planes
    this.DisPassNum = new Random().nextInt( bound: 50+1-10)+10;
    this.EmbPassNum = new Random().nextInt( bound: 50+1-10)+10;

}
```

Figure 4 - Plane constructor

In the construction of plane runnable process, the 6th plane is set to be the emergency plane with fuel shortage by setting its boolean variable emergencyLanding to “true”. The number if passengers that is going to embark on and disembark from the plane are also randomize in this stage and store inside the plane thread.

```java
// Start the flow of the plane after the plane''s thread is created and started
public void run() {
    System.out.println(id + ": Arrived at airport");

    requestLand();

    // Make the plane wait until it is landed
    synchronized (this) {
        while (!landStatus) {
            try {
                wait();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

    // After the plane landed successfully, the rest of the ground operation continues
    try {
        requestCoastandDock();
        Thread.sleep( millis: 2000);
        requestDisembarkPassenger();
        RequestRefuel();
        RequestCleanAndRefill();
        Thread.sleep( millis: 2000);
        ATC.EmbarkPassenger( plane: this);
        Thread.sleep( millis: 1000);
        requestUndock();
        Thread.sleep( millis: 1000);
        requestTakeOff();
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
}
```

Figure 5 - Plane run

After the plane thread is started, the overall flow of the simulation is all controlled by the plane thread itself.

3.3 Landing

```java
// Plane send request to ATC for permission to Land after arriving the airport
public void requestLand(){ 1 usage

    // Prompt different message according to the plane's emergency status
    if (emergencyLanding == true){
        System.out.println(id + ": Encountered fuel shortage and request for an emergency landing!");
    }else{
        System.out.println(id + ": Request for landing!");
    }

    try{
        Thread.sleep( millis: 1000);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }

    // Bring the flow to ATC's function
    ATC.confirmLanding( plane: this);
}
```

Figure 6- Plane requestLand()

After the plane arrived, it will request to land based on its emergency status, prompting different message to the ATC. Then, the flow will be passed on to ATC to handle the landing queue.

```java
// Confirm whether the plane can land or not after receiving plane's request to land
public static void confirmLanding(Plane plane) { 1 usage

    // Synchronize on the landingQueue to ensure thread safety, only one thread can access at a time
    synchronized (landingQueue) {

        // If the plane does not have an emergency, add it to the end of the landing queue
        if (!plane.getEmergencyStatus()){
            landingQueue.addLast( e: plane);
        }
        else{
            // If the plane has an emergency, add it to the front of the landing queue and notify the plane
            landingQueue.addFirst( e: plane);
            System.out.println("ATC: "+ plane.getId() + ": Request accepted. Successfully moved " + plane.getId() + " to the first in the queue.");
        }

        // Record the time the plane requested to land
        plane.setRequestTime(System.currentTimeMillis());

        // Notify any waiting plane threads that the state of the landingQueue has changed
        landingQueue.notifyAll();

        // Prompt message to plane when airport is full (3 or more planes on airport)
        if (planeOnGround >= 3){
            System.out.println("ATC: "+ plane.getId() + ": Airport is full! Please wait in queue for further notice.");
        }

        // Put the plane to wait when the runway is not available and airport is full
        while (planeOnGround >= 3 || runwaySemaphore.availablePermits() == 0) {
            try {
                landingQueue.wait();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }

        // After the waiting plane is notified, the first plane in the queue will land
        LandPlane( plane: landingQueue.poll());
    }
}
```

Figure 7 – ATC confirmLanding

After the plane request to land, the ATC will determine where will the plane be in queue based on its emergency status. If the plane is in emergency, ATC will allow the plane to cut queue and put it to the front of the landing queue. If the plane is not emergency, it will just be arranged to the back to the queue. The time of request of the plane is also recorded to calculate the waiting time of each plane. If the airport is full, where 3 planes is already in it or when runway is not available, the plane thread will be put in wait until further notify. If the landing is available, the first plane will be notified by the ATC to land. Synchronized block is also used on landingQueue to ensure that no multiple plane threads will call this method at the same time.

```java
// Plane Landing
public static void LandPlane(Plane plane){ 1 usage
    // Synchronize on the landingQueue to ensure thread safety, only one thread can access at a time
    synchronized (landingQueue){

        // Acquire the runway semaphore for the plane to land
        try{
            runwaySemaphore.acquire();
        }catch(InterruptedException e){
            e.printStackTrace();
        }
        System.out.println("ATC: "+ plane.getId() + ": Runway is available." + plane.getId() + " may land now.");
        try {
            Thread.sleep( millis: 1000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println(plane.getId() + ": Successfully landed!");

        // Record the time the plane successfully landed
        plane.setLandTime(System.currentTimeMillis());

        // Increment 1 to plane on ground
        planeOnGround++;

        // Set landstatus back to true
        plane.setLandStatus(true);

        // Calculate the waitingTime and add it into the statistic variable
        long waitingTime = plane.getLandTime() - plane.getRequestTime();
        statistic.addWaitingTime(waitingTime);

        // After landing successfully, release the semaphore for other plane to use
        runwaySemaphore.release();

        // Notify the planes in the landing queue
        landingQueue.notifyAll();
    }
}
```

Figure 8 - ATC landPlane

When the plane is landing, the runway semaphore is acquired and released after successfully landed. The land time is also recorded to conclude the total waiting time of the plane and added into the statistic. The notify function is the used to notify other waiting plane in the queue.

3.4 Gate Coasting and Docking

```java
// After landing successfully, plane send request to ATC for coasting and docking to available gate
public void requestCoastandDock() throws InterruptedException { 1 usage
    System.out.println(id+ ": Request to coast to available gate.");

    // Bring the flow to ATC's function
    ATC.ConfirmCoastAndDock( plane: this);
}
```

Figure 9 - Plane requestCoastandDock

After landed, plane will request to coast and dock to available gate, calling the ATC function.

```java
// Confirm the plane's request to coast and dock to available gate after landed
public static void ConfirmCoastAndDock(Plane plane) { 1 usage
    if (gateSemaphore.availablePermits() >=1){
        // Acquire the gate's semaphore for the plane to dock
        try{
            gateSemaphore.acquire();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        // Get the available gate using the assignGate() method and store the string into gate variable
        int availableGateIndex = assignGate();
        Gate gate = gates.get(availableGateIndex);


        if (availableGateIndex != -1) {
            // Assign the available gate to the plane to coast to and dock
            plane.setAssignedGate( assignedGate: gate.getId(), availableGateIndex);
            System.out.println("ATC: " + plane.getId() + ": Please proceed to coast and dock in " + gate + ".");
        }else{
            System.out.println("ATC: "+ plane.getId() + ": No gate available. Please wait for further notice."); //Should never reach here
            return;
        }

        try {
            Thread.sleep( millis: 1000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println(plane.getId() + ": Successfully coasted to " + gate + " and docked.");
    }
}
```

Figure 10 – ATC ConfirmCoastAndDock

Upon the plane's request, the ATC will acquire the gate semaphore and also assign an available gate to the plane using the assignGate function.

```java
private static final List<Gate> gates = Arrays.asList(new Gate( id: "Gate-1"), new Gate( id: "Gate-2"), new Gate( id: "Gate-3"));
```

```java
// Determine available gates from the array list and return the index number of the gate
public static int assignGate(){  1 usage
    for (int i = 0; i < gates.size(); i++) {
        if (gates.get(i).isAvailable()) {

            // Disable the gate so that other plane won't get the same gate at a same time
            gates.get(i).setAvailable(false);
            return i;
        }
    }
    return -1; // Should never reach here
}
```

Figure 11 – ATC assignGate

The assignGate function will go through the gate list and return the available gate's index. After assigning the gate to a plane, the gate's availability will be change to "false" so that other gate won't be assigned to the same gate.

3.5 Disembarking

```java
// After docking successful to a gate, plane request to ATC for disembarking passengers
public void requestDisembarkPassenger() throws InterruptedException {  1 usage
    System.out.println(id+ ": Request to disembark passenger. (Passenger Total: " + DisPassNum + ")");

    // Bring the flow to ATC's function
    ATC.DisembarkPassenger( plane: this);
}
```

Figure 12 - Plane requestDisembark

After successfully docked to a gate, the plane will then request to disembark passengers to the airport, directing the flow to ATC.

```java
// Confirm the plane's request to disembark passengers to the airport
public static void DisembarkPassenger(Plane plane){  1 usage
    System.out.println("ATC: "+ plane.getId() + ": Passenger from " + plane.getId() + " may disembark from the plane now. Please remember to " +
            "take all you belongings with you!");

    // Create passenger thread to run disembark function
    Passenger pass = new Passenger(plane, action: Action.Disembark);
    Thread disembarkpass = new Thread( task: pass);
    disembarkpass.start();

    try {
        // Use join() for the process to be completed before other process continues
        disembarkpass.join();
        Thread.sleep( millis: 1000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }

    System.out.println(plane.getId() + ": All " + plane.getDisPassNum() + " passengers from " + plane.getId() + " had successfully disembarked!");
}
```

Figure 13 - ATC DisembarkPassenger

Upon plane's request to disembark passengers, ATC will create thread for the disembarking passengers, by bring the action disembark into the runnable process from the Action enum class.

```java
// Constructor to initialize a passenger with the plane and an action (from the Action enum)
public Passenger(Plane plane, Action action) {  2 usages
    this.plane = plane;
    this.action = action;
}
```

Figure 14 - Passenger Constructor

```java
// Method to simulate disembarking of passengers from the plane
public void disembarkPlane(Plane plane) {  1 usage
    System.out.println(plane.getId() + ": Passengers are currently disembarking from " + plane.getId() + ".");

    // Simulate the time taken for each passenger to disembark (100 ms for each passenger)
    for (int i=0 ; i < plane.getDisPassNum(); i++){
        try {
            Thread.sleep( millis: 100);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Figure 15 - Passenger disembarkPlane

```java
// Enum class representing different actions that can be performed on a plane
public enum Action {  12 usages
    Disembark, // Action for passengers disembarking from a plane  2 usages
    Embark,    // Action for passengers boarding onto a plane  2 usages
    Cleaning,  // Action for cleaning a plane  2 usages
    Refill;    // Action for refilling supplies on a plane  2 usages
}
```

Figure 16 - Action enum class

```java
// Execute the passenger action based on the action when the thread is created
@Override
public void run() {
    if (action.equals(Action.Disembark))
    {
        disembarkPlane(plane);
    }
    else if (action.equals(Action.Embark))
    {
        boardPlane(plane);
    }
}
```

Figure 17 – Passenger's run

As stated in Figure 15, the time taken for the passengers to disembark is simulated where each passenger took 100 ms to disembark. A join function is also used so that the plane thread will wait for every passengers to disembark before moving on to other operations.

3.6 Refuel

```java
// After plane successfully disembark all passengers to the airport, plane will request to ATC for refueling
public void RequestRefuel() throws InterruptedException {  1 usage
    System.out.println(id + ": Request for refueling!");

    // Bring the flow to ATC's function
    ATC.refuelPlane( plane: this);
}
```

Figure 18 - Plane RequestRefuel

After every passengers successfully disembarked, plane will request for refuelling services, then pass the flow to ATC.

```java
// Confirm the plane's request to refuel plane
public static void refuelPlane(Plane plane) throws InterruptedException {  1 usage

    // Create thread for refuelling truck
    RefuelingTruck refueling = new RefuelingTruck(plane);
    Thread refuel = new Thread( task: refueling);

    // Acquire the refuel semaphore to refuel the plane
    refuelingSemaphore.acquire();
    try {
        // Try to start the thread and use join() to wait for the refuelling process to finish
        System.out.println("ATC: "+ plane.getId() + ": Request for refuelling approved.");
        refuel.start();
        refuel.join();
    } finally {
        // Release the refuel semaphore after the refuelling process is done
        refuelingSemaphore.release();
    }
}
```

Figure 19 - ATC refuelPlane

```java
public class RefuelingTruck implements Runnable{  2 usages
    private Plane plane;  3 usages
    private final Lock refuelingLock = new ReentrantLock(); // Lock to ensure exclusive access to the refueling process  2 usages

    // Constructor to initialize the RefuelingTruck with a specific plane
    public RefuelingTruck(Plane plane) {  1 usage
        this.plane = plane;
    }

    @Override
    public void run() {

        // Acquire the lock before refueling
        refuelingLock.lock();
        try {
            // Simulate refueling
            System.out.println("Refueling worker: Currently refuelling " + plane.getId() + ".");
            try{
                Thread.sleep( millis: 2000);
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
            System.out.println("Refueling worker: "+ plane.getId() + " had successfully refueled.");
        } finally {
            // Release the lock after refueling is complete
            refuelingLock.unlock();
        }
    }
}
```

Figure 20 - RefuellingTruck class

As shown in Figure 19, upon the plane's request, a thread is created for the refuellingTruck process, bring in the requesting plane. After acquiring the refuel semaphore, the thread is started, and join is used so that the plane will wait for the refuelling process to complete before moving on. As shown in Figure 20, a refuelingLock is used so that it is available for one thread only at a time, as there are only one refuelling truck in the airport. After the process is done, the lock will be unlocked, allowing other plane to acquire the refuel truck.

3.7 Cleaning and Refilling Services

```java
// After plane's refueling process is done, plane will request for cleaning and refilling supplies
public void RequestCleanAndRefill(){  1 usage
    System.out.println(id+ ": Request for Cleaning and Supplies Refilling services.");

    // Bring the flow to ATC's function
    ATC.CleaningAndRefilling( plane: this);
}
```

Figure 21 - Plane RequestCleanAndRefill

After refuelling process is done, the plane will request to ATC for cleaning and supplies refilling services.

```java
// Confirm the plane's request to clean the plane and refill supplies to the plane
public static void CleaningAndRefilling(Plane plane){  1 usage
    System.out.println("ATC: "+ plane.getId() + ": Request accepted. Cleaning and Refilling process on " + plane.getId() + " will now occur simultaneously.");

    // Create two workers object to clean and refill simultaneously
    Worker worker1 = new Worker( id: 1,plane,  action: Action.Cleaning);
    Worker worker2 = new Worker( id: 2,plane,  action: Action.Refill);

    // Create threads  for both clean and refill
    Thread clean = new Thread( task: worker1);
    Thread refill = new Thread( task: worker2);

    // Start the thread
    clean.start();
    refill.start();

    // Use join() for both process to be completed before other process continues
    try{
        clean.join();
        refill.join();
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
}
```

Figure 22 - ATC CleaningAndRefilling

Upon the plane's request, two worker thread will be created and started, each with different actions, which is cleaning and supplies refilling, using the action enum class as shown in Figure 16.

```java
// Method to simulate cleaning the plane
public void Cleaning(Plane plane){  1 usage
    System.out.println("Worker " + id + ": Currently cleaning " + plane.getId() + ".");

    try{
        Thread.sleep( millis: 1000);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }

    System.out.println("Worker " + id + ": Cleaning process on " + plane.getId() + " had been successfully executed.");
}

// Method to simulate refilling supplies on the plane
public void Refill(Plane plane){  1 usage
    System.out.println("Worker " + id + ": Currently refilling supplies on " + plane.getId() + ".");

    try{
        Thread.sleep( millis: 1000);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }

    System.out.println("Worker " + id + ": The supplies on " + plane.getId() + " had been successfully refilled.");
}
```

Figure 23 - Worker Class Cleaning and Refill

```java
// Execute the worker's action based on the action when the thread is created
@Override
public void run() {
    if (action.equals(Action.Cleaning))
    {
        Cleaning(plane);
    }
    else if (action.equals(Action.Refill))
    {
        Refill(plane);
    }
}
```

Figure 24 – Worker's run

As shown in Figure 23, the time for both cleaning and supplies refilling are set to 1 seconds each. As shown in Figure 22, a join function is used to ensure that both process will be completed before the plane move on to other operations.

3.8 Embark

```java
// Embark passengers onto the plane
public static void EmbarkPassenger(Plane plane){ 1 usage
    System.out.println("ATC: "+ plane.getId() + ": Passengers will now embark onto " + plane.getId() + ". (Passenger total: " + plane.getEmbPassNum() + ")");

    // Create passengers thread to run the boardPlane function
    Passenger pass = new Passenger(plane, action: Action.Embark);
    Thread embarkpass = new Thread( task: pass);
    embarkpass.start();

    try {
        // Use join() for the process to be completed before other process continues
        embarkpass.join();
        Thread.sleep( millis: 1000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }

    System.out.println("ATC: All " + plane.getEmbPassNum() + " passengers has successfully embark onto " + plane.getId() + ". Please be prepared to take off.");

    // Add the passengers boarded to the total passengers boarded statistics
    statistic.addPassengersBoarded( count: plane.getEmbPassNum());
}
```

Figure 25 - ATC EmbarkPassenger

```java
// Method to simulate boarding of passengers onto the plane
public void boardPlane(Plane plane) { 1 usage
    System.out.println(plane.getId() + ": Passengers are currently boarding onto " + plane.getId() + ".");

    // Simulate the time taken for each passenger to board (100 ms for each passenger)
    for (int i=0 ; i < plane.getEmbPassNum(); i++){
        try {
            Thread.sleep( millis: 100);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Figure 26 - Passenger boardPlane

After the services are done, the ATC will arrange passengers to board onto the plane. The entire process is similar to 3.5, and after every passengers successfully board onto the plane, the number of passengers boarded are added to the total number of passengers boarded.

3.9 Undock

```java
// After all passengers successfully board onto the plane, plane will request to ATC for undocking from the gate
public void requestUndock(){ 1 usage
    System.out.println(id + ": Request to undock from " + assignedGate + ".");

    // Bring the flow to ATC's function
    ATC.ConfirmUndock( plane: this);
}
```

Figure 27 - Plane requestUndock

After the passengers successfully boarded onto the plane, the plane will request to undock from the assigned gate to prepare for take-off.

```java
// Confirm the plane's request to undock from the gate after every ground operation is done
public static void ConfirmUndock(Plane plane) {  1 usage
    System.out.println("ATC: " + plane.getId() + ": Request accepted." + plane.getId() + " may now undock from " + plane.getGate() + ".");

    try{
        Thread.sleep( millis: 1000);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }

    System.out.println(plane.getId() + ": Successfully undocked from " + plane.getGate() + ".");

    // Release the gate after the plane successfully undocked from the gate
    releaseGate(plane.getAssignedGateIndex());
}
```

Figure 28 - ATC ConfirmUndock

```java
// Method to release gate after the planes undock successfully
public static void releaseGate(int gateIndex) {  1 usage

    // Set the availability of gate back to true and release gate semaphore
    gates.get(gateIndex).setAvailable(true);
    gateSemaphore.release();
    System.out.println("ATC: " + gates.get(gateIndex).getId() + " is now available.");
}
```

Figure 29 - ATC releaseGate

Upon plane's request, ATC will release the gate as shown in Figure 28 and 29. The plane's assigned gate index will then be brought into the releaseGate function, and the gate will be set to available for other plane to coast and docked to.

3.10 Take Off

```java
// After plane successfully undock from the gate, plane will request to take off
public void requestTakeOff() throws InterruptedException {  1 usage
    System.out.println(id + ": All ground operations had been successfully done. Request to take off!" );

    // Bring the flow to ATC's function
    ATC.Takeoff( plane: this);
}
```

Figure 30 - Plane requestTakeOff

After every ground operation is done, the plane will request to take off.

```java
// Confirm the plane's request to takeoff from the airport
public static void Takeoff(Plane plane) throws InterruptedException {  1 usage
    // Synchronize on the landingQueue to ensure thread safety, only one thread can access at a time
    synchronized (landingQueue){
        // Acquire the runway semaphore for the plane to take off
        try {
            runwaySemaphore.acquire();
        } catch (InterruptedException e){
            e.printStackTrace();
        }
        System.out.println("ATC: " + plane.getId() + ": Runway is available. " + plane.getId() + " may now coast to runway and take off.");
        try{
            Thread.sleep( millis: 2000);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        System.out.println(plane.getId() + ": Successfully took off! Thanks for the service.");
        // Decrement 1 for the plane on ground
        planeOnGround--;
        // Set landstatus back to false
        plane.setLandStatus(false);
        // Release the runway semaphore after the plane successfully took off
        runwaySemaphore.release();
        // Notify the planes in the landing queue
        landingQueue.notifyAll();
        // Increment the number fo planes served
        statistic.incrementPlanesServed();
        // If all planes are served and the gates are all empty, end recording the simulation time and print the statistics
        if (statistic.planesServed.get() == TOTAL_PLANES && statistic.allGatesEmpty(gates)) {
            statistic.endSimulation();
            statistic.printStatistics();
        }
    }
}
```

Figure 31 - ATC Takeoff

Upon plane's request, ATC will try to acquire the runway semaphore for the plane to take off. After the plane successfully took off, the semaphore will be release and notifyAll is used to notify the plane in landing queue about the changes to the number of planes in airport. As this is the last step for the simulation, if all 6 planes had been served successfully and all gates are empty, the ATC will print the statistics, indicating the end of the simulation.

3.11 Statistics

```java
public class Statistics {  2 usages
    private final List<Long> waitingTimes = new ArrayList<>();  7 usages
    final AtomicInteger planesServed = new AtomicInteger( initialValue: 0);  3 usages
    private final AtomicInteger passengersBoarded = new AtomicInteger( initialValue: 0);  2 usages
    private long startSimulationTime;  2 usages
    private long endSimulationTime;  2 usages
    private static final DecimalFormat decfor = new DecimalFormat( pattern: "0.00");  1 usage
```

Figure 32 - Statistic Variables

As shown in Figure 32, AtomicInteger is used for planesServed and passengersBoarded to ensure thread-safe operations, where only one thread can modify the value at a time. This provides a simple and efficient way to handle thread-safe increments and additions without requiring explicit synchronization. The ArrayList waitingTimes is used to store the waiting times for each plane. An decimal format is also created to 2 decimal places for the average waiting time.

```java
// Method to record the start time of the simulation
public void startSimulation() {  1 usage
    this.startSimulationTime = System.currentTimeMillis();
}

// Method to record the end time of the simulation
public void endSimulation() {  1 usage
    this.endSimulationTime = System.currentTimeMillis();
}

// Method to add the waiting time of the planes to the list
public void addWaitingTime(long waitingTime) {  1 usage
    synchronized (waitingTimes) {
        waitingTimes.add(waitingTime);
    }
}

// Method to increment the number of planes served
public void incrementPlanesServed() {  1 usage
    planesServed.incrementAndGet();
}

// Method to add to the total number of passengers boarded
public void addPassengersBoarded(int count) {  1 usage
    passengersBoarded.addAndGet( delta: count);
}
```

Figure 33 - Statistics methods

```java
// Method to check if all gates are empty
public boolean allGatesEmpty(List<Gate> gates) {
    for (Gate gate : gates) {
        if (!gate.isAvailable()) {
            return false;
        }
    }
    return true;
}
```

Figure 34 - Statistics allGatesEmpty

Figure 34 shows a boolean method to check whether all gates are empty as it is one of the condition before printing the statistics.

```java
// Method to print the statistics at the end of the simulation
public void printStatistics() {  1 usage
    // Calculate the total simulation time
    long totalSimulationTime = endSimulationTime - startSimulationTime;
    System.out.println("\n==================================Statistics================================== ");
    System.out.println("Total simulation time: " + totalSimulationTime/1000 + " seconds.");
    System.out.println("Number of planes served: " + planesServed.get());
    System.out.println("Number of passengers boarded: " + passengersBoarded.get());
    // Synchronize on the waitingTimes to ensure thread safety, only one thread can access at a time
    synchronized (waitingTimes) {
        if (!waitingTimes.isEmpty()) {
            long maxWaitTime = waitingTimes.stream().max( comparator: Long::compare).orElse( other: 0L);
            long minWaitTime = waitingTimes.stream().min( comparator: Long::compare).orElse( other: 0L);
            double avgWaitTime = waitingTimes.stream().mapToLong( mapper: Long::longValue).average().orElse( other: 0.0);

            System.out.println("Maximum waiting time: " + maxWaitTime/1000 + " seconds.");
            System.out.println("Average waiting time: " + decfor.format( number: avgWaitTime/1000) + " seconds");
            System.out.println("Minimum waiting time: " + minWaitTime/1000 + " seconds.");
            System.out.println("==================================End of Simulation==================================");

            System.exit( status: 0);
        }
    }
}
```

Figure 35 - Statistics printStatistics

As shown in Figure 35, all time units printed in the statistics are converted to seconds as the default unit is milliseconds. The calculation of maximum, minimum and average wait times will provide a default value of 0 for maximum and minimum, and 0.0 for average if the waitingTimes is empty, which shall not happen. The decimal format stated earlier is used for the average waiting times so that it only gives value in 2 decimal places.

4.0 Output

4.1 Simulation Process

For the simulation process, I will be focusing on a “Plane-1” for example.

```
Plane-1: Arrived at airport
Plane-1: Request for landing!
ATC: Plane-1: Runway is available.Plane-1 may land now.
```

Figure 36 - Plane Arrive and Request Landing

```
Plane-1: Successfully landed!
Plane-1: Request to coast to available gate.
ATC: Plane-1: Please proceed to coast and dock in Gate-1.
```

```
Plane-1: Successfully coasted to Gate-1 and docked.
```

Figure 37 - Coast and Dock

```
Plane-1: Request to disembark passenger. (Passenger Total: 45)
ATC: Plane-1: Passenger from Plane-1 may disembark from the plane now. Please remember to take all you belongings with you!
Plane-1: Passengers are currently disembarking from Plane-1.
```

Figure 38 - Disembark Passenger

```
Plane-1: All 45 passengers from Plane-1 had successfully disembarked!
Plane-1: Request for refueling!
ATC: Plane-1: Request for refuelling approved.
Refueling worker: Currently refuelling Plane-1.
```

```
Refueling worker: Plane-1 had successfully refueled.
```

Figure 39 - Successful Disembark and Refuel Request

```
Plane-1: Request for Cleaning and Supplies Refilling services.
```

```
ATC: Plane-1: Request accepted. Cleaning and Refilling process on Plane-1 will now occur simultaneously.
Worker 1: Currently cleaning Plane-1.
Worker 2: Currently refilling supplies on Plane-1.
Worker 2: The supplies on Plane-1 had been successfully refilled.
Worker 1: Cleaning process on Plane-1 had been successfully executed.
```

Figure 40 - Cleaning and Supplies Refilling

```
ATC: Plane-1: Passengers will now embark onto Plane-1. (Passenger total: 47)
Plane-1: Passengers are currently boarding onto Plane-1.
```

```
ATC: All 47 passengers has successfully embark onto Plane-1. Please be prepared to take off.
```

Figure 41 - Embark Passengers

```
Plane-1: Request to undock from Gate-1.
ATC: Plane-1: Request accepted.Plane-1 may now undock from Gate-1.
```

```
Plane-1: Successfully undocked from Gate-1.
ATC: Gate-1 is now available.
```

Figure 42 - Undock

```
Plane-1: All ground operations had been successfully done. Request to take off!
ATC: Plane-1: Runway is available. Plane-1 may now coast to runway and take off.
```

```
Plane-1: Successfully took off! Thanks for the service.
```

Figure 43 - Take Off

4.2 Emergency Case

For the emergency case, I set "Plane-6" as the plane that faces fuel shortage and require emergency landing.

```
Plane-6: Arrived at airport
Plane-6: Encountered fuel shortage and request for an emergency landing!
```

Figure 44 - Request Landing

```
ATC: Plane-6: Request accepted. Successfully moved Plane-6 to the first in the queue.
ATC: Plane-6: Airport is full! Please wait in queue for further notice.
```

Figure 45 - Request Accepted and Cut Queue

```
Plane-1: Successfully took off! Thanks for the service.
ATC: Plane-6: Runway is available.Plane-6 may land now.
Plane-6: Successfully landed!
```

Figure 46 - Land first

As shown in Figure 42, after the first plane (Plane-1) took off, the ATC allows Plane-6 to land first, even though it came last. This shows that the cut queue of Plane-6 is successful.

4.3 Statistics

```
=====================================Statistics=====================================
Total simulation time: 52 seconds.
Number of planes served: 6
Number of passengers boarded: 176
Maximum waiting time: 27 seconds.
Average waiting time: 12.36 seconds
Minimum waiting time: 1 seconds.
==================================End of Simulation==================================
```

Figure 47 - Statistics Result

5.0 Conclusion

The Asia Pacific Airport operations simulation was successful in meeting all of the objectives, proving the efficiency of the airport's management system in limited circumstances. We were able to mimic aircraft landing, servicing, and take-off on a single runway while staying within the airport's capacity restriction of having no more than three aircraft on the property at once by putting in place a Java-based model.

The concurrent processes of passenger embarkation and disembarkation, as well as the unique refuelling event, were managed properly, ensuring that activities ran smoothly and continuously. The simulation also included a congested situation in which the airport is already full, and there are planes waiting in queue, while a plane with had to make emergency landing due to a fuel shortage. This scenario put the airport's capacity to manage high-pressure situations to the test, as well as its ability to prioritise key events while maintaining overall efficiency.

Key performance measures were gathered and analysed, including maximum, average, and minimum plane wait times, as well as the number of planes served, and passengers boarded. The sanity checks at the end of the simulation revealed that all gates were indeed empty, indicating that the implemented procedures were proper.

Overall, this simulation offered useful information on the Asia Pacific Airport's operational dynamics. The findings emphasised the need of excellent resource management and coordination in supporting the smooth flow of airport operations. The successful completion of this simulation not only fits present operational requirements, but also provides a framework for future additions to improve airport efficiency and safety.

6.0 References

LinkedList (Java Platform SE 8). (2024, April 4). https://docs.oracle.com/javase/8/docs/api/java/util/LinkedList.html

Pankaj. (2022, August 3). *AtomicInteger in Java*. DigitalOcean. https://www.digitalocean.com/community/tutorials/atomicinteger-java

GeeksforGeeks. (2024b, March 11). *Semaphore in Java*. GeeksforGeeks. https://www.geeksforgeeks.org/semaphore-in-java/

GeeksforGeeks. (2024e, June 18). *ArrayList in Java*. GeeksforGeeks. https://www.geeksforgeeks.org/arraylist-in-java/

Jenkov, J. (n.d.). *Java DecimalFormat*. https://jenkov.com/tutorials/java-internationalization/decimalformat.html